

Skybound Slayer 企画書

3DアクションRPG プログラム設計・実装仕様資料

1. 基本情報

タイトル	Skybound Slayer	ジャンル	3D アクション RPG
開発環境	C++ / DxLib / Effekseer / Visual Studio 2022	対応 OS	Windows
開発体制	個人制作 1名	開発期間	約2ヶ月
総ファイル数	約90ファイル		

2. ゲーム概要

3Dフィールドを探索しながら敵を倒し、討伐で得た資金でステータスを強化してボス討伐を目指すアクションゲームです。通常攻撃やジャンプ攻撃、スライドアタックといった多彩なアクションと無敵時間を活用した回避、ショップでのキャラクタービルド要素を組み込んでいます。

3. 操作方法

入力キー	割り当てアクション
W A S D	移動
Space	回避
マウス左クリック	攻撃
E	攻撃方法切り替え
← → / R	アイテム選択 / 使用
Enter / P / Esc	ショップ表示 / ポーズ / 終了確認

4. シーン管理設計

SceneManagerクラスが全体のシーン遷移を一元的に管理しています。管理対象にはタイトル画面、ルール説明、設定画面、メインゲーム、リザルト画面が含まれます。シーン変更時は予約関数を經由し、フレームの終端で一括更新を行うことで、処理実行中の不正なアクセスやリソース破棄を防ぐ設計としています。

5. オブジェクト基底クラス設計

3D空間に存在する全オブジェクトはObject3Dクラスを継承します。共通の識別タグ、衝突イベント時の各種コールバック関数、地形追従処理、および半透明オブジェクト描画時のZソート処理を備えています。

```
// オブジェクト基底クラスの主要インターフェース
class Object3D { public: virtual ~Object3D() = default;
virtual void Update(float delta_time) = 0; virtual void Draw() const = 0;
virtual void OnEnter(Collider* collider, Object3D* other) {}
virtual void OnTrigger(Collider* collider, Object3D* other) {}
virtual void OnExit(Collider* collider, Object3D* other) {}
void TerrainFollow(const Stage& stage);
protected: Tag3D tag_{Tag3D::kNone}; Vector3 position_{0.0f, 0.0f, 0.0f};
};
```

- **Player3D**: プレイヤーの操作、移動物理、戦闘状態を管理
- **Enemy3D**: 近接型やモンスター、ボスなど各敵キャラクターの共通基底
- **Stage / SkyBox / Rock**: 地形および背景オブジェクトの描画と判定管理
- **Coin**: ドロップアイテムの移動と取得判定処理

6. 自作当たり判定システム

Enter、Trigger、Exitの3ステートによる判定システムを自作し、ColliderManagerで一括処理を行っています。

クラス名	役割・用途
Collider	位置、判定半径、衝突中リストを保持する基底クラス
SphereCollider	球体判定。攻撃判定や広範囲索敵に使用
CapsuleCollider	カプセル判定。キャラクター本体の物理衝突に使用
ColliderManager	全コライダーを保持し毎フレームの衝突を一括計算

コライダーをコンポーネントとして親オブジェクトにアタッチする構成を採用し、1つのキャラクターが攻撃用・体当たり用・索敵用などの複数の判定を個別に保持できるようにしています。また、破棄フラグによる遅延解放処理を導入し、ループ実行中のメモリ破棄による不具合を回避しています。

7. サービスロケーターパターン

Masterクラスを全体の統一アクセス窓口とし、各マネージャーへ効率的にアクセスできる構成を採用しました。階層構造が深くなった際のポイントのバケツリレーを解消しています。

```
// Masterクラスを介したマネージャー参照例 Master::sound_manager_->PlayBGM("game"); Master::enemy_m  
anager_->SpawnWave();
```

8. プレイヤー制御とステータス連携

Player3Dでは通常攻撃、ジャンプ攻撃、スライド攻撃の各状態を列挙型で管理し、クールダウンおよび回避中の無敵フレーム制御を行っています。ショップで購入したパラメータ強化値は専用変数へ即座に反映されます。

9. 課題解決と今後の改善点

課題: アニメーションの重複と入力割り込み

動作中に次の入力が入った際、モーションが途中で切れて重複再生される問題が発生しました。ステート側で現在の再生状態と遷移タイミングをチェックする条件判定を追加し、不正な割り込みを阻止しました。

課題: ショップ用アイテムデータの増加に伴うコード肥大化

価格や上昇値などのデータを直接記述していたためコードが増大していました。項目を外部CSVファイルへ切り出し、起動時に読み込む構造へ変更することで可読性とデータ調整の効率を向上させました。

今後の改善:

メモリ管理やクラスの設計が甘いので、今後のゲーム制作では改善していきたい